Astro326 Lab 2

Group F: Samuel Jediah Gultom1, Tai Chung2, Adrien Bonansea3

October 2024

1Department of Astronomy and Astrophysics, University of Toronto, Contact: samuel.gultom@mail.utoronto.ca
2Department of Astronomy and Astrophysics, University of Toronto, Contact: tai.chung@mail.utoronto.ca
3Department of Astronomy and Astrophysics, University of Toronto, Contact: adrien.bonansea@mail.utoronto.ca

1 Abstract

This lab explores the relationship between thermal radiation and temperature using an Airspy radio receiver. We measured power spectra across various temperatures, from 1.5°C to 80.8°C, as well as liquid nitrogen and dry ice, collecting up to 100 million data points per set. By minimizing radio frequency interference, we analyzed data distributions and performed Chi-squared fits and use these to find relationships between temperature power spectra and the measuring devices gain.

2 Introduction

In astronomy, various sensors and detection devices are crucial for recording electromagnetic waves from radio to infrared wavelengths. Thermal radiation is the electromagnetic energy that all material gives off as long as it contains so form of energy (has a temperature above 0 kelvin). The larger the temperature, the shorter the wavelengths emitted and the more recordings we can get.

First we have to ask ourselves why we would use radio astronomy and not rely for optical detectors. Although this topic can be spoken about at length, we will only discuss the main reason for the use of thermal detectors over these optical ones. When trying to detect radio waves optically we run into the issue that there are many overpowering sources of radio waves, which interfere with the results. The Biggest reason for this is the light coming from Cosmic microwave background having shifted into radio waves (Barron et al. 2022).

For this paper, we detect thermal radiation of various liquids at different temperatures. The range

we will encompass is from about 100 C to the temperature of liquid nitrogen which is very low temperature. We will use an AirSpy radio receiver which will allow us to collect sets of 100 million data points at each measured temperature chosen. We will first discuss the specifications and functions of this device, followed by an explanation of the math used during analysis. Finally we will describe our experimental set up, the reduction of data we had to use, how we managed uncertainties, and finally we will discuss trends we see in the spectra and other data presented at each temperatures.

3 Data and Observation

3.1 AirSpy Detectors

To understand how the Airspy device works, it is best to follow a diagram for easier understanding. We will follow a simplified diagram that is shown in Figure 1:

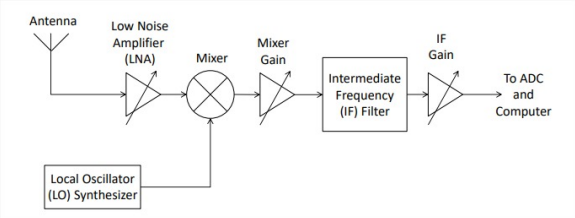

Figure 1: Diagram of how the AirSpy detector takes in and converts data

After submerging the antenna in the liquid of choice at a temperature of choice, we capture incoming radio signals. The Airspy then converts the incoming frequencies into electrical data which is amplified by the LNA (low noise amplifier). An added benefit of the LNA is that is reduces noise while amplifying signals, giving us overall better data to work with. The signals then move to the mixer and mixer gain where frequency specific signals from the LO (local oscillator) are added which results in a current that combines both intermediate and desired frequencies. These are then moved to the IF (intermediate frequency filter) and IF gain which makes the data easier to process by isolating the desired frequency components and reducing unwanted signals before getting transferred onto your computer.
The Airspy does thus process with an efficiency of about 5 million data points recorded per second. We used this to have data sets at each temperature of 100 million data points.

3.2 Distribution Methods

In this lab we will mainly use 2 different distribution methods for analysis, the Normal distribution and the Chi-squared distribution χ^2

3.2.1 Normal distribution

The normal distribution, described by equation 1 below, is a widely used statistical method known for its symmetry around the mean.

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2} \tag{1}$$

This distribution forms the characteristic "bell curve" shape, where most values are concentrated near the mean, and the probability of observing values decreases as they move further from the center. One of the notable features of the Gaussian distribution is that approximately 68% of values fall within one standard deviation of the mean, 95% within two, and 99.7% within three, making this a very useful model for natural variations. (Cowan 2023). Furthermore, the larger the data set the more accurate the distribution, in other words we will have a very accurate distribution even if we use a sample of the data collected.

3.2.2 Chi-squared distribution χ^2

The chi-squared distribution, described by equation 2 below, continuous probability distribution that describes random variables formed from summing the squares of values. x is considered to be the random variable and k the degrees of freedom.

$$f(x;k) = \frac{1}{2^{k/2}\Gamma\left(\frac{k}{2}\right)} x^{\frac{k}{2}-1} e^{-x/2} \quad \text{for } x \geq 0 \tag{2}$$

Where x is the random variable and k is the degrees of freedom.

3.3 Fitting Method

We will be using the linear least squares fitting method which allows us to determine the best fine line within a set of data points. TO do this, we will use the Python Scipy library which contains the curve fit function. the chosen equation for the curve-fit can be seen with equation 3 below

$$y = mx + b \tag{3}$$

4 Data Reduction and Methods

As discussed previously, to collect the data we submerge the antenna into the liquid of choice. The liquids we ended up using was water at 6 different temperatures and liquid nitrogen. In addition, we used dry ice where we placed the antenna on the dry ice instead of submerging it and measure the at room temperature by leaving the antenna out an playing a measurement.. The chosen temperatures

for the water were: 80.8 C, 65.5 C, 48.5 C, 38.1 C, 13.6 C, and 1.5 C. Between measurements we let the antenna we made sure to have some time after placing the antenna in a new liquid so the antenna could adjust and have less faulty measurements.

Despite this, when making a histogram of data and plotting data points we decided to take a sample of 1000 points as this is more than enough to make an accurate normal distribution. This also avoids using the first couple thousand points which had a tendacy to be skewed and quite off from the other points.

When taking data, we took data sets of 100 million data points for all exept the nitrogen and dry ice, where we took a data set of 10 million to reduce computational load. All data sets had to be taken on the same day, with the same airspy, with the same compute at the same frequency to any reduce possible environmental changes or change defects in within devices.

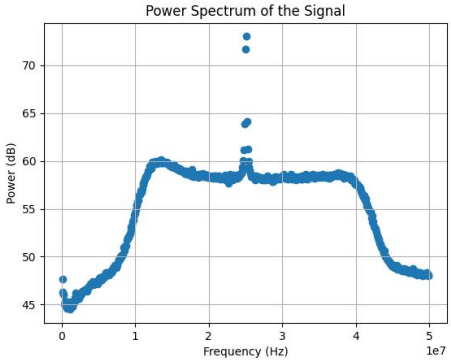

Figure 2: Power spectrum with RFI

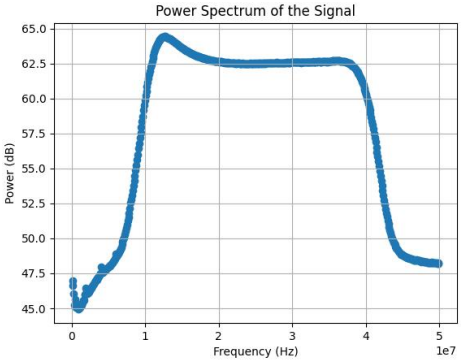

Figure 3: Power spectrum without RFI

To avoid complications with the RFI in the power spectrum, we tested multiple frequencies and found one with little to no RFI interference. As can be seen from figure 2 and 3, figure 2 has a lot of RFI due to the points drifting up instead of being continuous on the curve. On the other hand figure 3 shows the same power spectrum but at a frequency of 609Hz which ended up being the optimal frequency and the one we ended up doing all of our temperature measurements with. Additionally, these example of power spectrums are both taken at 1.5C to have consistently in showing the effect of RFI in power spectrums.

An additional reduction that was made on the power spectrum is quite small but was skewing with the scale of the graphs. the 1st point of every power spectrum was much higher than every other point in the spectrum at every instance. To make sure that the graphs are showing needless whitespace and decreasing visibility, we remove this point at the beginning of every power spectrum.

5 Data Analysis and Modeling

In this section we will first analyze the data using a scatter plot to see that there is indeed an evenly distributed signal across the figure (Figure 4). This makes sure that the antenna did in-fact take good data and we can use the data set. We then graph this small set of data into a histogram and overlay the normal distribution over the top of it (Figure 5).
In this case, we are analyzing the dataset which was taken at room temperature. If we would like to analyze the other data sets, we just have to switch the file in the code and rerun the code snippets. On the other hand, we will compare the various spectra of all temperatures later on in the paper to try to find trends that relate temperature the spectra.

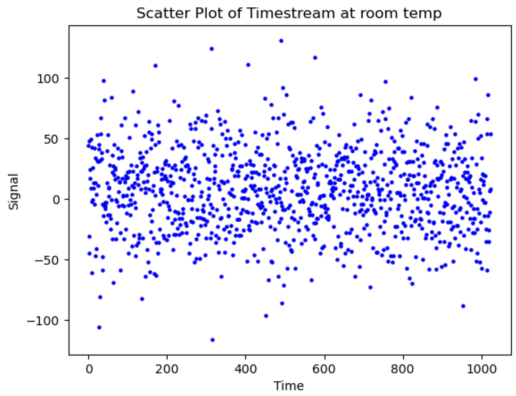

Figure 4: Scatter plot of of last 1000 points at room temp

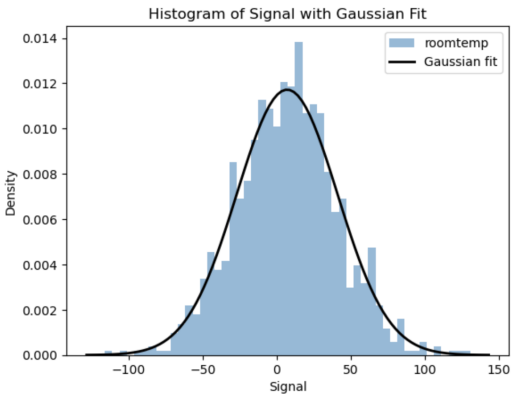

Figure 5: Histogram and distribution of of last 1000 points at room temp

We graph the the power distribution and Chi-squared fits at n = 1, 2, 4, 10, and 100 (Figure 6). As we can see, the chi-squared fits vary quite a lot which is expected. We also notice that at $n = 100$ the line is not visible. If we isolate the n = 100 line we see that this is due to the gradient of the line being equal to 0.

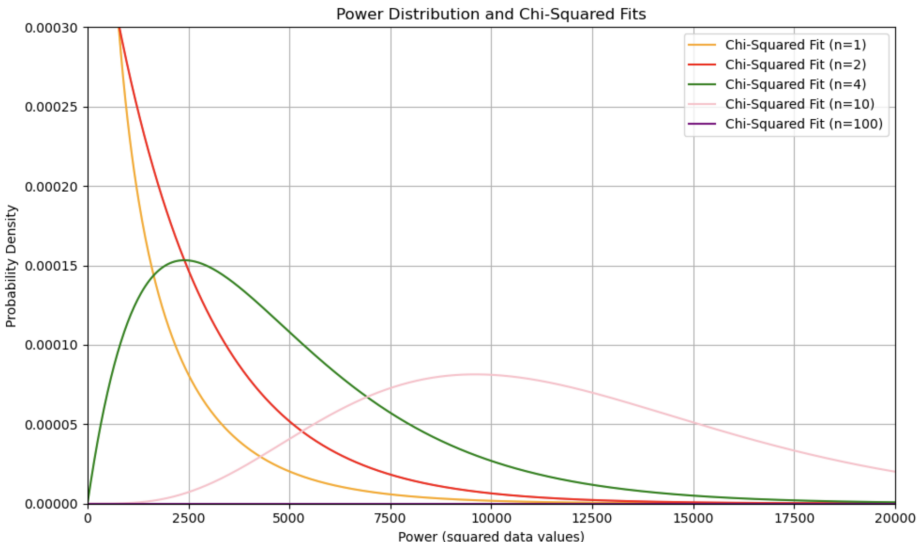

Figure 6: Power distributions with various chi squared values

In Figure 7 we graph the power spectrum of for the data set at room temperature.

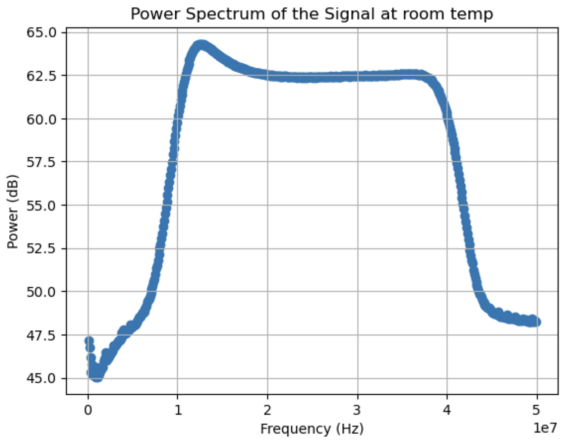

Figure 7: power spectrum at room temperature

The next part of the experiment analysis the difference between power spectrum and trends when if the temperature increases or decreases.

As we can see below figure 8 and 9 look quite similar. Figure 8 is the full power spectrum of all the various data sets. We notice that most of the lines are at the same points when the spectrum is increasing or decreasing, making the differences less visible.

We use a python command to create figure 9 from figure 9, limiting the y axis so we can see the important information and the changes in peaks of the spectrum. Through this figure, we ca clearly

see that as the temperature increases, the overall spectrum shifts up by a small amount.

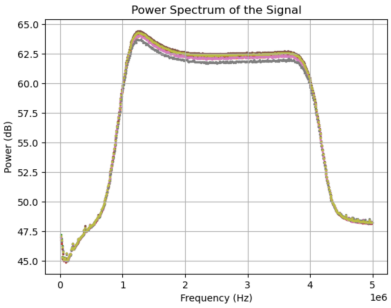

Figure 8: Full power spectrum

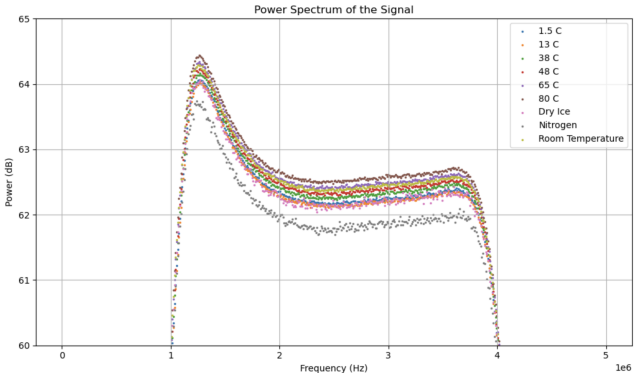

Figure 9: Limited power spectrum

Furthermore, Table 1 below shows the load Temperatures, Variances, and Estimated Uncertainties and shows us a similar trend. We obtain their uncertainties by

Table 1: Load Temperatures, Variances, and Estimated Uncertainties of datasets

Load Temperature (K)	**Variance (bits²)**	**Estimated Uncertainty (bits²)**
353.95	1183.49	0.00628
338.65	1159.77	0.00615
321.65	1135.94	0.00603
311.25	1119.49	0.00594
299.35	1151.05	0.00611
286.75	1088.27	0.00577
274.65	1098.03	0.00583
194.65	1090.46	0.00579
77.35	1009.66	0.00536

Using the data from the data below, we can use the linear fit from section 3.3 of this paper. By curve fitting the points we with their uncertainties, we are able to see figure 10.
Using the fit and the points, we can then obtain the residuals for each point, as shown on figure 11.

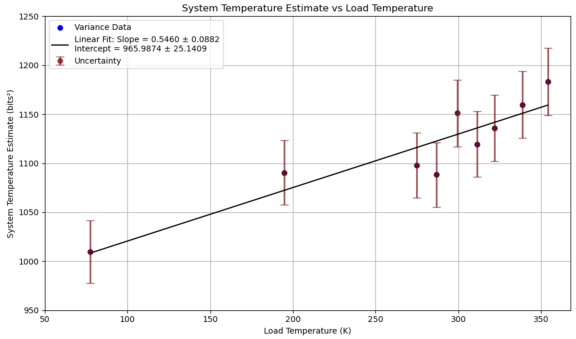

Figure 10: Linear fit of varince and load temp

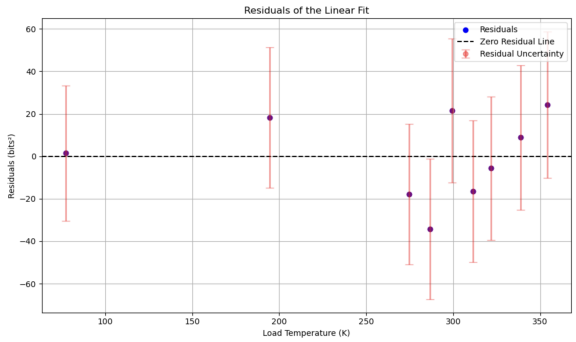

Figure 11: Residuals of Figure 10

Through both figures 10 and 11, we see that the linear fit a good fir for the system vs load temperatures due to the linear fit being within the uncertainties and the residuals being at most around 2.5% of the points themselves. Additionally, we find that the slope in Figure 10 is 0.5460 ± 0.0882 and the intercept is 965.9874 ± 25.1409. To calculate the gain, we use:

$$G = \frac{\Delta V}{\Delta T} = \text{slope} \tag{4}$$

In other words, the slope is equal to the gain, meaning that the gain is:

$$G = slope = 0.5460 \pm 0.0882 \tag{5}$$

We can then conclude that a positive change in temperature will leave to an increase in variance of the system and small changes in T can lead to large changes in variance

6 Discussion and Conclusion

In this lab, we were able to successfully demonstrated the relationship between the power spectrum of thermal radiation and temperature. By using the Airspy receivers at various temperatures, we were able to determine that if the temperature increased, the peak wavelength that is emitted is shorter, leading to a shift up in energy and power. This also provides a compelling aregument for the principles of black body radiation. We were then able to able to calculate then gain to be 0.5460 ± 0.0882 by using the variance of every dataset and using a curve fit through the Scipy python library to find a general trend. The trend showed us that if teh temperature increaces the variance also changes greatly. The uncertainties calculated in this paper were quite small which is partially thanks to our choice of single frequency to remove most of the RFI. We also removed certain data points which skewed the data and took samples to not include data where the antenna was no yet calibrated to the temperature. However, we did not solve all the issue, as we cannot simulate perfect conditions for every result. To enhance the robustness of future experiments, we suggest plotting the gain and receiver temperature across a broader range of frequencies. Even with RFI we collecting more data points for both at other frequencies and more extreme temperatures (warmer ones). We also were not able to complete the full experiment due to time constraints. in this section we would measure the spectral properties and plot the gain vs the temperature to get the a function of frequency which tells us the receivers efficiency and noise characteristics. In conclusion, despite not being able to measure the Airspy noise and efficiency, our study provides us with a relationship between power spectra and temperature and how temperatures affects the Gain the device.

7 References

Cowan, G. 2023, Statistical Data Analysis (Clarendon)

8 Appendix

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm
data = np.fromfile('group_F3_room_26_2.dat',dtype='int16')-2**11
small_set = data[-1024:]
time = range(len(small_set))  # or use actual time stamps if available

plt.scatter(time, small_set, s=5, c='blue', alpha=1)  # 's' controls point size
plt.xlabel('Time')
plt.ylabel('Signal')
plt.title(f'Scatter Plot of Timestream at room temp')
plt.show()

# Plot the histogram
plt.hist(small_set, bins=50, alpha=0.5, density=True, label="roomtemp")

# Calculate mean and standard deviation
mean = np.mean(small_set)
std = np.std(small_set)

# Generate values for the Gaussian curve
xmin, xmax = plt.xlim()  # Get the current limits of the x-axis
x = np.linspace(xmin, xmax, 100)
p = norm.pdf(x, mean, std)

# Plot the Gaussian curve
plt.plot(x, p, 'k', linewidth=2, label='Gaussian fit')

# Add labels and title
plt.xlabel('Signal')
plt.ylabel('Density')
plt.title('Histogram of Signal with Gaussian Fit')
plt.legend()

# Show the plot
```

```
plt.show()
mean = np.mean(data)
variance = np.var(data)
print(mean)
print(variance)
mean_small = np.mean(small_set)
variance_small = np.var(small_set)
print(mean_small)
print(variance_small)
n = 1024
data = data[:len(data) // n * n]
data = data.reshape(-1, 1024)
fft_res = np.fft.fft(data, axis=1)
freq = np.fft.fftfreq(1024, d=1/100000000)
power_spectrum = np.abs(fft_res[:, :1024//2])**2
average_spectrum = np.mean(power_spectrum, axis=0)
spectrum_db = 10 * np.log10(average_spectrum)
plt.scatter(freq[:1024//2][1:], spectrum_db[1:]) # plot frequencies vs power in dB
plt.xlabel('Frequency (Hz)')
plt.ylabel('Power (dB)')
plt.title('Power Spectrum of the Signal at room temp')
plt.grid(True)
plt.show()

import numpy as np
import matplotlib.pyplot as plt
import scipy.stats as ss

# Load and adjust the data
data = np.fromfile('group_F3_room_26_2.dat',dtype='int16')-2**11

# First 1000 Points on a Scatter Plot
data_26_scatter = data[-1000:]
x_26_scatter = np.linspace(0, 999, 1000)

data_26 = data[-1000000:]
```

```
power_26 = data_26 ** 2


# Chi-Squared Distribution
x_26_chi = np.linspace(0, max(power_26), 1000)
chi2_26 = ss.chi2.pdf(x_26_chi, df=1, loc=0, scale=1200)


# Aggregated Power and Chi-Squared Fits
power_26 = power_26[:len(power_26) // 10 * 10]  # Ensure divisibility


# n = 1
power_26_n1 = power_26.reshape(-1, 1).sum(axis=1)
#plt.hist(power_26_n2, bins=30, density=True, color='red', alpha=0.8, label='n = 2')
chi2_26_n2 = ss.chi2.pdf(x_26_chi, df=1, loc=0, scale=1200)
plt.plot(x_26_chi, abs(chi2_26_n2))

# n = 2
power_26_n2 = power_26.reshape(-1, 2).sum(axis=1)
#plt.hist(power_26_n2, bins=30, density=True, color='red', alpha=0.8, label='n = 2')
chi2_26_n2 = ss.chi2.pdf(x_26_chi, df=2, loc=0, scale=1200)
plt.plot(x_26_chi, abs(chi2_26_n2))

# n = 4
power_26_n4 = power_26.reshape(-1, 4).sum(axis=1)
#plt.hist(power_26_n4, bins=30, density=True, color='green', alpha=0.6, label='n = 4')
chi2_26_n4 = ss.chi2.pdf(x_26_chi, df=4, loc=0, scale=1200)
plt.plot(x_26_chi, abs(chi2_26_n4))

# n = 10
power_26_n10 = power_26.reshape(-1, 10).sum(axis=1)
#plt.hist(power_26_n10, bins=30, density=True, color='pink', alpha=0.4, label='n = 10')
chi2_26_n10 = ss.chi2.pdf(x_26_chi, df=8, loc=0, scale=1200)
plt.plot(x_26_chi, abs(chi2_26_n10))

# n = 10
```

```
power_26_n10 = power_26.reshape(-1, 10).sum(axis=1)
#plt.hist(power_26_n10, bins=30, density=True, color='pink', alpha=0.4, label='n = 8')
chi2_26_n10 = ss.chi2.pdf(x_26_chi, df=10, loc=0, scale=1200)
plt.plot(x_26_chi, abs(chi2_26_n10))

# n = 10
power_26_n100 = power_26.reshape(-1, 8).sum(axis=1)
#plt.hist(power_26_n100, bins=30, density=True, color='blue', alpha=0.4, label='n = 100')
chi2_26_n100 = ss.chi2.pdf(x_26_chi, df=100, loc=0, scale=1200)
plt.plot(x_26_chi, abs(chi2_26_n100))

plt.xlim(0,20000)
plt.ylim(0,0.0003)
plt.legend()
plt.show()


import numpy as np
import matplotlib.pyplot as plt

# Load and adjust the data
data_1_5 = np.fromfile('group_F_1_5.dat', dtype='int16') - 2**11
data_13 = np.fromfile('group_F_13_6.dat', dtype='int16') - 2**11
data_38 = np.fromfile('group_F_38_1.dat', dtype='int16') - 2**11
data_48 = np.fromfile('group_F_48_5.dat', dtype='int16') - 2**11
data_65 = np.fromfile('group_F_65_5.dat', dtype='int16') - 2**11
data_80 = np.fromfile('group_F_80_8.dat', dtype='int16') - 2**11
data_ice = np.fromfile('group_F_dry_ice_10m.dat', dtype='int16') - 2**11
data_nit = np.fromfile('group_F_nitrogen_10m.dat', dtype='int16') - 2**11
data_room = np.fromfile('group_F3_room_26_2.dat', dtype='int16') - 2**11

all_data = [data_1_5, data_13, data_38, data_48, data_65, data_80, data_ice, data_nit, data_room]
labels = ['1.5 C', '13 C', '38 C', '48 C', '65 C', '80 C', 'Dry Ice', 'Nitrogen', 'Room Temperatur

plt.figure(figsize=(10, 6))  # Optional: Set figure size

for data, label in zip(all_data, labels):
```

```
    n = 1024
    data = data[:len(data) // n * n]
    data = data.reshape(-1, 1024)
    fft_res = np.fft.fft(data, axis=1)
    freq = np.fft.fftfreq(1024, d=1/10000000)
    power_spectrum = np.abs(fft_res[:, :1024 // 2]) ** 2
    average_spectrum = np.mean(power_spectrum, axis=0)
    spectrum_db = 10 * np.log10(average_spectrum)

    # Plot frequencies vs power in dB and include label
    plt.scatter(freq[:1024 // 2][1:], spectrum_db[1:], s=2, label=label)

# Finalize the plot
plt.xlabel('Frequency (Hz)')
plt.ylabel('Power (dB)')
plt.title('Power Spectrum of the Signal')
plt.ylim(60, 65)
plt.grid(True)
plt.legend()  # Display the legend
plt.tight_layout()  # Optional: Adjust layout
plt.show()
for data in all_data:
    n = 1024
    data = data[:len(data) // n * n]
    data = data.reshape(-1, 1024)
    fft_res = np.fft.fft(data, axis=1)
    freq = np.fft.fftfreq(1024, d=1/10000000)
    power_spectrum = np.abs(fft_res[:, :1024//2])**2
    average_spectrum = np.mean(power_spectrum, axis=0)
    spectrum_db = 10 * np.log10(average_spectrum)
    plt.scatter(freq[:1024//2][1:], spectrum_db[1:], s = 2) # plot frequencies vs power in dB
    plt.xlabel('Frequency (Hz)')
    plt.ylabel('Power (dB)')
    plt.title('Power Spectrum of the Signal')

    plt.grid(True)
plt.show()
```

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

# Read in Data
data_files = [
    'group_F_80_8.dat',
    'group_F_65_5.dat',
    'group_F_48_5.dat',
    'group_F_38_1.dat',
    'group_F3_room_26_2.dat',
    'group_F_13_6.dat',
    'group_F_1_5.dat',
    'group_F_dry_ice_10m.dat',
    'group_F_nitrogen_10m.dat'
]

# Load the data and subtract the offset
data = [np.fromfile(file, dtype='int16') - 2**11 for file in data_files]

# Calculate Variance and Load Temperatures
k = 273.15  # Offset for converting to Kelvin
var = [np.var(d) for d in data]
load_temp = [80.8 + k, 65.5 + k, 48.5 + k, 38.1 + k, 26.2 + k, 13.6 + k, 1.5 + k, -78.5 + k, -195.8

# Calculate uncertainties
uncert = [item / np.sqrt(3.552 * 10**10) for item in var]
uncert_std = np.sqrt(var)

# Create the scatter plot with error bars
plt.figure(figsize=(10, 6))
plt.scatter(load_temp, var, label='Variance Data', color='blue')
plt.errorbar(load_temp, var, yerr=uncert_std, fmt='o', alpha=0.7,
             color='maroon', elinewidth=2, capsize=5, label='Uncertainty')
# Linear Fit
def linear_model(x, a, b):
```

```
    return a * x + b

popt, pcov = curve_fit(linear_model, load_temp, var)
a_, b_ = popt

# Extracting the uncertainties of the slope and intercept
slope_uncertainty = np.sqrt(pcov[0, 0])  # Uncertainty of the slope
intercept_uncertainty = np.sqrt(pcov[1, 1])  # Uncertainty of the intercept

# Plotting the linear fit line
new_load_temp = np.array(load_temp)
plt.plot(new_load_temp, linear_model(new_load_temp, *popt), color='black',
         label='Linear Fit: Slope = {:.4f} ± {:.4f}\nIntercept = {:.4f} ± {:.4f}'.format(a_, slope
# Finalizing the plot
plt.title('System Temperature Estimate vs Load Temperature')
plt.xlabel('Load Temperature (K)')
plt.ylabel('System Temperature Estimate (bits²)')
plt.ylim(bottom=950)
plt.ylim(top=1250)
plt.xlim(left=50)
plt.grid()
plt.legend(loc='upper left')
plt.tight_layout()
plt.show()

# Print values
print("Load Temperatures (K):", load_temp)
print("Variances (bits²):", var)
print("Estimated Uncertainties (bits²):", uncert)

# Calculate residuals
predicted_variance = linear_model(new_load_temp, *popt)
residuals = var - predicted_variance

# Create the residuals plot
plt.figure(figsize=(10, 6))
plt.scatter(load_temp, residuals, color='blue', label='Residuals')
```

```
plt.axhline(0, color='black', linestyle='--', label='Zero Residual Line')

# Add error bars for residuals based on uncertainties
plt.errorbar(load_temp, residuals, yerr=uncert_std, fmt='o', alpha=0.4,
             color='red', elinewidth=2, capsize=5, label='Residual Uncertainty')

# Finalizing the residuals plot
plt.title('Residuals of the Linear Fit')
plt.xlabel('Load Temperature (K)')
plt.ylabel('Residuals (bits²)')
plt.grid()
plt.legend(loc='upper right')
plt.tight_layout()
plt.show()
```